

In-Class Assignment 7: Identifying and Correcting Dimensionality Errors in CNNs

SHASHANK B G

20022954

explaining each step and why the dimensions change the way they do.

1. Initial Input Understanding

- CIFAR-10 images are 32x32 pixels with 3 color channels (RGB)
- Therefore, the input shape is 32x32x3
- The first two numbers (32,32) represent height and width
- The last number (3) represents the color channels (RGB)

2. First Convolutional Layer (Conv1)

- Filter size: 3x3
- Stride: 1 (default)
- Padding: 'valid' (no padding)
- Input: 32x32x3
- Calculation:

$$\text{output_size} = (\text{input_size} - \text{filter_size} + \text{stride}) / \text{stride}$$

$$\text{output_size} = (32 - 3 + 1) / 1 = 30$$

- Output: 30x30x32
 - 30x30 because each 3x3 filter operation reduces dimension by 2
 - 32 is the number of filters (creating 32 feature maps)

3. First Max Pooling Layer (Pool1)

- Window size: 2x2
- Stride: 2
- Input: 30x30x32

Calculation:

$$\text{output_size} = (\text{input_size} - \text{pool_size} + \text{stride}) / \text{stride}$$

$$\text{output_size} = (30 - 2 + 2) / 2 = 15$$

- Output: 15x15x32
 - Each 2x2 window with stride 2 halves the dimensions
 - Channel depth (32) remains unchanged in pooling

4. Second Convolutional Layer (Conv2)

- Filter size: 3x3

- Stride: 1
- Input: 15x15x32

Calculation:

$$\text{output_size} = (15 - 3 + 1) / 1 = 13$$

- Output: 13x13x64
 - 13x13 from the convolution operation
 - 64 is the new number of filters

5. Problem: Original Second Max Pooling Layer (Pool2)

- Window size: 3x3
- Stride: 3
- Input: 13x13x64

Calculation:

$$\text{output_size} = (13 - 3 + 3) / 3 = 4.33 \text{ that is almost equal to } 4$$

- Output: 4x4x64
- Issues:
 1. Uneven division (13 doesn't divide evenly by 3)
 2. Too aggressive reduction (13x13 → 4x4)
 3. Information loss due to large stride

6. Solution: Corrected Second Max Pooling Layer (Pool2)

- Window size: 2x2
- Stride: 2
- Input: 13x13x64

Calculation:

$$\text{output_size} = (13 - 2 + 2) / 2 = 6.5 \text{ that is equal to } 6$$

- Output: 6x6x64
- Benefits:
 1. More gradual reduction (13x13 → 6x6)
 2. Better preservation of spatial information
 3. Follows standard CNN practices

7. Why This Correction Matter

- Better Feature Preservation:
 - 2x2 pooling examines smaller regions, preserving more local features
 - Stride of 2 ensures no overlap between pooling windows

- Results in more useful features for classification
- Improved Network Architecture:
 - More balanced reduction in spatial dimensions
 - Maintains more spatial information for the dense layers
 - Follows proven CNN design patterns (like VGG network)
- Training Benefits:
 - More stable gradients during backpropagation
 - Better feature hierarchy learning
 - Potentially better classification accuracy

Error Identification in the code:-

The error is in the second MaxPooling2D layer (Pool2) which uses a 3x3 filter with implicit stride of 3.

This causes two issues:

1. Excessive information loss due to large filter size and stride
2. Poor dimension reduction ratio compared to standard practices

The correct code is:-

```
def build_cnn_model(input_shape, num_classes):
    model = models.Sequential()
    # First Convolutional Layer
    model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=input_shape, name='Conv1'))
    model.add(layers.MaxPooling2D((2, 2), name='Pool1'))
    # Second Convolutional Layer - CORRECTED POOLING
    model.add(layers.Conv2D(64, (3, 3), activation='relu', name='Conv2'))
    model.add(layers.MaxPooling2D((2, 2), name='Pool2')) # Changed from (3,3) to (2,2)
    # Flatten and Dense Layers
    model.add(layers.Flatten(name='Flatten'))
    model.add(layers.Dense(64, activation='relu', name='Dense1'))
    model.add(layers.Dense(num_classes, activation='softmax', name='Output'))
    # Compile the Model
    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
```

```
metrics=['accuracy']  
  
)  
  
return model
```

Justification for Correction:-

The correction to use a 2x2 MaxPooling filter with stride 2 is better because:

1. Consistent downsampling: 2x2 pooling with stride 2 is the standard practice in CNNs as it provides a good balance between dimension reduction and information preservation.
2. Better information retention: The 2x2 filter preserves more spatial information compared to the 3x3 filter, which is crucial for feature detection.
3. More appropriate output dimensions: The corrected model produces 6x6 feature maps before flattening, giving the dense layers more spatial information to work with compared to the original 4x4 feature maps.
4. Standard architectural pattern: Most successful CNN architectures (like VGG) use 2x2 max pooling with stride 2 as it has proven to be effective for feature extraction and dimension reduction.